

1. Java

```
public class FacturaProcessor {
    public static void procesarFactura(Factura f, Cliente c, Config config) {
        // 1. Validaciones iniciales (15 líneas)
        if (f == null || c == null) throw new IllegalArgumentException("Datos
inválidos");
        if (f.getItems().isEmpty()) throw new IllegalStateException("Factura vacía");
        if (c.getId() <= 0) throw new IllegalArgumentException("Cliente inválido");

        // 2. Cálculo de subtotales (12 líneas)
        double subtotal = 0;
        for (Item item : f.getItems()) {
            subtotal += item.getCantidad() * item.getPrecio();
            if (item.getCantidad() < 0) throw new IllegalArgumentException("Cantidad
negativa");
        }

        // 3. Aplicar descuentos por cliente (18 líneas)
        double descuento = 0;
        if (c.getTipo().equals("VIP") && subtotal > 100000) descuento = subtotal * 0.15;
        else if (c.getTipo().equals("Regular") && subtotal > 50000) descuento = subtotal
* 0.10;
        else if (c.getAntigüedad() > 5 && subtotal > 20000) descuento = 2000;

        // 4. Calcular impuestos (10 líneas)
        double baseImponible = subtotal - descuento;
        double iva = baseImponible * 0.19;
        double total = baseImponible + iva;

        // 5. Logging extenso (12 líneas)
        System.out.println("Factura #" + f.getId());
        System.out.println("Cliente: " + c.getNombre());
        System.out.println("Subtotal: $" + subtotal);
        System.out.println("Descuento: $" + descuento);
        System.out.println("IVA: $" + iva);
        System.out.println("TOTAL: $" + total);
    }
}
```

```
        // 6. Guardar en base de datos (5 líneas simuladas)
        try { guardarFactura(f, total); } catch (Exception e) { logError(e); }
    }
}
```

2. Python

```
class SistemaVentas:
    clientes = []
    productos = []
    facturas = []

    def registrarCliente(self, nombre, email, telefono, direccion, tipo):
        cliente = {
            'id': len(self.clientes) + 1, 'nombre': nombre, 'email': email,
            'telefono': telefono, 'direccion': direccion, 'tipo': tipo
        }
        self.clientes.append(cliente)
        self.enviarBienvenida(email)
        print(f"Cliente {nombre} registrado")

    def agregarProducto(self, codigo, nombre, precio, stock):
        producto = {'codigo': codigo, 'nombre': nombre, 'precio': precio, 'stock':
stock}
        self.productos.append(producto)

    def procesarVenta(self, cliente_id, items):
        factura = {'id': len(self.facturas) + 1, 'cliente_id': cliente_id, 'items':
items}

        subtotal = sum(item['cantidad'] * item['precio'] for item in items)
        descuento = self.calcularDescuento(cliente_id, subtotal)
        iva = (subtotal - descuento) * 0.19
        total = subtotal - descuento + iva
        factura['total'] = total
        self.facturas.append(factura)
        self.actualizarStock(items)
        self.enviarFactura(cliente_id, factura)

    def calcularDescuento(self, cliente_id, subtotal):
        cliente = next(c for c in self.clientes if c['id'] == cliente_id)
        if cliente['tipo'] == 'VIP' and subtotal > 100000: return subtotal * 0.15
        return 0

    def actualizarStock(self, items):
        for item in items:
```

```
        prod = next(p for p in self.productos if p['codigo'] == item['codigo'])
        prod['stock'] -= item['cantidad']

    def enviarBienvenida(self, email): print(f"Bienvenida a {email}")
    def enviarFactura(self, cliente_id, factura): print(f"Factura enviada
#{factura['id']}")
```

3. JavaScript

```
class ProcesadorPagos {
  procesarPago(metodo, monto, datos) {
    switch(metodo) {
      case 'tarjeta':
        if (!datos.numero || !datos.cvv || !datos.expiracion) {
          throw new Error('Datos tarjeta incompletos');
        }
        return this.validarTarjeta(datos.numero, datos.cvv, datos.expiracion,
monto);

      case 'paypal':
        if (!datos.email || !datos.password) {
          throw new Error('Credenciales PayPal faltantes');
        }
        return this.autenticarPaypal(datos.email, datos.password, monto);

      case 'transferencia':
        if (!datos.banco || !datos.numero_cuenta) {
          throw new Error('Datos transferencia inválidos');
        }
        return this.procesarTransferencia(datos.banco, datos.numero_cuenta,
monto);

      case 'efectivo':
        return {aprobado: true, metodo: 'efectivo', monto};

      case 'cripto':
        if (!datos.wallet || !datos.moneda) {
          throw new Error('Wallet cripto requerida');
        }
        return this.procesarCripto(datos.wallet, datos.moneda, monto);

      default:
        throw new Error(`Método ${metodo} no soportado`);
    }
  }

  validarTarjeta(num, cvv, exp, monto) { /* 10 líneas */ }
```

```
autenticarPaypal(email, pass, monto) { /* 8 líneas */ }  
// ... más métodos  
}
```

4. C#

```
public class BancoService {
    public decimal Transferir(Cuenta origen, Cuenta destino, decimal monto) {
        // Usa 40 líneas de métodos de CUENTA
        if (!origen.TieneFondosSuficientes(monto)) return 0;
        var saldoActual = origen.GetSaldoActual();
        var nuevoSaldo = saldoActual - monto;
        origen.SetSaldoTemporal(nuevoSaldo);

        var comision = destino.CalcularComisionTransferencia(monto);
        var saldoDestino = destino.GetSaldoActual();
        destino.AgregarSaldo(monto - comision);
        destino.ActualizarHistorialTransferencias(origen.Id, monto);

        origen.RegistrarMovimiento("Transferencia salida", -monto);
        destino.RegistrarMovimiento("Transferencia entrada", monto - comision);

        origen.ValidarLimitesDiarios(monto);
        destino.ValidarPoliticasyAntilavado(monto);
        return monto - comision;
    }
}

public class Cuenta {
    public decimal GetSaldoActual() => saldo;
    public bool TieneFondosSuficientes(decimal monto) => saldo >= monto;
    // ... 20+ métodos usados por BancoService
}
```

5. Python

```
class Formulario {
    def crearUsuario(nombre, apellido, email, telefono,
                    dir_calle, dir_numero, dir_ciudad, dir_pais, dir_cp,
                    fact_calle, fact_numero, fact_ciudad, fact_pais, fact_cp):
        # Validar dirección personal (15 líneas)
        if not nombre or len(nombre) < 2: raise ValueError("Nombre inválido")
        if not dir_calle or not dir_ciudad: raise ValueError("Dirección incompleta")

        # Validar dirección fiscal (15 líneas)
        if not fact_calle or not fact_ciudad: raise ValueError("Dir. fiscal incompleta")

        # Crear usuario con 3 direcciones (20 líneas)
        usuario = Usuario(nombre, apellido, email, telefono)
        usuario.direccion = Direccion(dir_calle, dir_numero, dir_ciudad, dir_pais,
dir_cp)
        usuario.direccion_fiscal = Direccion(fact_calle, fact_numero, fact_ciudad,
fact_pais, fact_cp)
        return usuario

    def actualizarUsuario(id, nombre, apellido, email, telefono,
                        dir_calle, dir_numero, dir_ciudad, dir_pais, dir_cp,
                        fact_calle, fact_numero, fact_ciudad, fact_pais, fact_cp):
        # MISMA validación repetida (15 líneas duplicadas)
}
```

6. Java

```
public boolean procesarPedido(Pedido p) {
    if (p != null) {
        if (p.getCliente() != null) {
            if (p.getCliente().isActivo()) {
                if (p.getItems() != null && !p.getItems().isEmpty()) {
                    if (p.getItems().stream().allMatch(i -> i.getStock() > 0)) {
                        if (this.validarCredito(p.getCliente())) {
                            if (this.validarInventario(p.getItems())) {
                                if (this.validarReglasNegocio(p)) {
                                    if (this.enviarConfirmacion(p.getCliente())) {
                                        if (this.actualizarInventario(p.getItems())) {
                                            if (this.generarFactura(p)) {
                                                // ;12 niveles de indentación!
                                                return true;
                                            }
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }
    return false;
}
```

7. C++

```
class ServicioA {
    bool validarEmail(string email) {
        if (!email.contains("@")) return false;
        if (email.length() < 5) return false;
        // 25 líneas de validación regex, sanitización, etc.
        return true;
    }
};

class ServicioB {
    bool validarEmail(string email) { // DUPLICADO EXACTO
        if (!email.contains("@")) return false;
        if (email.length() < 5) return false;
        // 25 líneas IDENTICAS
        return true;
    }
};

class ServicioC { /* MISMO CÓDIGO COPIADO */ }
```